

Introduction to Computer Networks

Recitation 02: Error Correcting Codes

Gabriel Domingues
gm@mail.tau.ac.il

Today's recitation: Error Correcting Codes

- Basic coding theory
- Repetition code
- Hamming codes
- Cyclic redundancy check (CRC)

- Error Correcting Code: Encode a message in a larger string in a way that can correct/detect errors
- Let $\text{dist}(c, c')$ denote the Hamming distance, i.e., the number of places that c and c' differ
- Send c , but received c' . The number of errors is $\text{dist}(c, c')$
- Notation (n, k, d)
 - Codeword length (encoded string length): n
 - Message length (number of information bits): k
 - Encode : $\{0, 1\}^k \rightarrow C \subseteq \{0, 1\}^n$
 - Code distance: $d = \min_{\substack{c, c' \in C \\ c \neq c'}} \text{dist}(c, c')$.
- A (n, k, d) code can:
 - Detect at most $d - 1$ errors
 - Correct at most $\lfloor (d - 1)/2 \rfloor$ errors
 - Correct at most $d - 1$ erasures

- The repetition code: send the message three times
- For every bit, majority wins (2-out-of-3 voting)
 - $(3k, k, 3)$ code
 - Can correct at most 1 error in each bit
 - Can detect at most 2 errors in each bit
 - Rate: $R = k/n = 1/3$
- Triple modular redundancy (TMR)



- The Hamming code with parameter r is a $(2^r - 1, 2^r - r - 1, 3)$ code
 - Perfect code: every word at distance at most 1 to a codeword
 - Rate: $R = k/n = 1 - r/(2^r - 1) \rightarrow 1$ as $r \rightarrow \infty$
- The Hamming code words are $C = \{y \in \{0, 1\}^{2^r - 1} \mid Hy = 0\}$ where H :
 - Lists in the columns all vectors in $\{0, 1\}^r \setminus \{0^r\}$.

$$H = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix} \text{ for } r = 3$$

- For message $x \in \{0, 1\}^{2^r - r - 1}$, encode by adding parity bits $\{p_i\}_{i=1}^r$ at positions 2^{i-1}

$$y = (p_1, p_2, x_1, p_3, x_2, x_3, x_4) \text{ for } r = 3$$

- Syndrome decoding: Let $s = Hy$, where y is the received word
 - if $s = 0$, there are no errors
 - if $s \neq 0$, the single error is at the index of s in the columns of H
- Hamming bound: $2^k \leq 2^n / (1 + n)$ for $d = 3$

- Consider polynomials in $GF(2)$
 - Coefficients are bits $\{0, 1\}$
 - Addition of bits is XOR
- Fix a polynomial $G(x) = x^n + \sum_{i=0}^{n-1} G_i x^i$
- Let $M(x) = \sum_{i=0}^{k-1} M_i x^i$
- Compute $R(x) = (M(x) \cdot x^n) \bmod G(x)$
- Encoding: $(M(x), R(x))$
- Variations:
 - Endianness: LSB or MSB
 - Preprocessing/Postprocessing (e.g., add an offset)
- Distance d depends strongly on $G(x)$

- Polynomial division is slow – checking for errors must be fast
- Optimizing remainder calculation
 - Initialize $R(x) \leftarrow M(x) \cdot x^n$
 - Repeat while $\deg(R) \geq n$:

$$R(x) \leftarrow R(x) + G(x) \cdot x^{\deg R(x) - n}$$

- Maintain the remainder using n bits
 - Initialize $R(x) \leftarrow M(x) \cdot x^{n-1-\deg M(x)}$ where $\deg M(x) < n$
 - Repeat $\deg M(x) + 1$ times:

$$R(x) \leftarrow \begin{cases} R(x) \cdot x + G(x) & \text{if } R(x)[x^{n-1}] = 1 \\ R(x) \cdot x & \text{otherwise} \end{cases}$$

- Streaming: Compute with each new byte of the message
 - $R_{old}(x) = (M_{old}(x) \cdot x^n) \bmod G(x)$
 - $R_{old}(x) = R_{high}(x) \cdot x^{n-8} + R_{low}(x)$ where $\deg R_{high}(x) < 8$ and $\deg R_{low}(x) < n - 8$.
 - $M_{new} = M_{old}(x) \cdot x^8 + B(x)$
 - Compute:

$$\begin{aligned} R_{new}(x) &= (M_{new}(x) \cdot x^n) \bmod G(x) \\ &= (R_{old}(x) \cdot x^8 + B(x) \cdot x^n) \bmod G(x) \\ &= (\underbrace{(R_{high}(x) + B(x)) \cdot x^n}_{\deg < 8}) \bmod G(x) + R_{low}(x) \cdot x^8 \end{aligned}$$

- The byte encoding is often implemented by a table lookup (Sarwate algorithm)
 - $\text{CRCTable}[P] = (P(x) \cdot x^n) \bmod G(x)$ for $\deg P(x) < 8$.
 - CRCTable has $2^8 = 256$ entries.

Code example (CRC32 LSB)

```
static uint32_t CRCTable[256] = {0};

void crc32_init(uint32_t G) {
    for (size_t i = 0; i != 256; ++i) {
        uint32_t r = i;
        for (size_t j = 0; j != 8; ++j)
            r = (r >> 1) ^ ((r & 1) ? G : 0);
        CRCTable[i] = r;
    }
}

uint32_t crc32(uint8_t data[], size_t n) {
    uint32_t crc = 0;
    for (size_t i = 0; i != n; ++i)
        crc = (crc >> 8) ^ CRCTable[(crc & 0xFF) ^ data[i]];
    return crc;
}
```

Questions?